



**WYDZIAŁ FIZYKI
I INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Bujalski

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: aplikacje mobilne

Numer albumu: 386012

Rotacyjnie inwariantne sieci neuronowe

Praca magisterska

wykonana pod kierunkiem

dr Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych, WFiIS UŁ

Łódź 2025

Spis treści

Wstęp	3
Cel i motywacja pracy	3
Opis pracy	4
Zakres tematyczny	5
Ujęte w zakresie	5
Poza zakresem	5
Artefakty pracy	6
Organizacja pracy	6
Podstawy teoretyczne	7
Wprowadzenie do sieci konwolucyjnych (CNN)	7
Inwariancja translacyjna i rotacyjna	7
Problemy z rotacyjną inwariancją w klasycznych CNN	7
Przegląd literatury (np. E(2)-Equivariant CNNs, CyCNN)	7
Opis zbiorów danych	7
MNIST (cyfry odręczne)	7
GTSRB Gray (znaki drogowe w odcieniach szarości)	7
GTSRB RGB (znaki drogowe)	7
LEGO (obiekty 3d rzutowane na 2d)	7
Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów	7
Architektury modeli	7
Standardowe CNN	7
Rotacyjnie inwariantne sieci (np. CyResNet, CyVGG, G-CNN)	7
Transformacje polarne: linearpolar vs logpolar	7
Implementacja i środowisko eksperymentalne	7
Python, PyTorch	7
Struktura projektu	7
Automatyzacja: skrypty trenowania, testowania, ewaluacji	7
Obsługa GPU, Docker, WSL	7
Organizacja logów, modeli, confusion matrixów	7
Eksperymenty	8
Scenariusze trenowania/testowania (opis JSON)	8
Pomiar skuteczności (accuracy, macierze pomyłek)	8
Śledzenie metryk: średnia, mediana, odchylenie standardowe	8
Analiza skuteczności względem rotacji	8
Ranking modeli	8
Porównanie wyników	8
CyCNN vs klasyczne CNN	8
VGG vs CyVGG	8

Resnet vs CyResNet	8
CyVGG vs CyResNet	8
Wpływ transformacji (linearpolar vs logpolar)	8
Wydajność na różnych zbiorach	8
Wnioski	8
Skuteczność rotacyjnych architektur	8
Wnioski z automatyzacji i systematyzacji ewaluacji	8
Propozycje dalszych badań	8
Bibliografia	8
Aneks	8
Listingi kodów	8
Dodatkowe wykresy, tablice wyników	8

Wstęp

Obrazy otaczają nas z każdej strony: od zdjęć ze smartfonów, przez monitoring miejski, katalogi produktów i systemy kontroli jakości na liniach produkcyjnych, po systemy wspomagania jazdy. Choć współczesne modele rozpoznawania obrazu radzą sobie bardzo dobrze, w praktyce bywają wrażliwe na pozornie drobne zmiany takie jak obrócenie obiektu o kilkanaście stopni czy niewielki przechył kamery. To, co dla człowieka jest naturalne i natychmiast rozpoznawalne (znak drogowy pod kątem, cyfra obrócona na kartce), dla klasycznej konwolucyjnej sieci neuronowej bywa problemem. Rdzeń trudności to brak naturalnej inwariantności względem rotacji: standardowe CNN-y „z definicji” lepiej radzą sobie z przesunięciami niż z obrotami.

W ostatnich latach pojawiło się kilka dróg domknięcia tej luki. Jedną to poszerzanie danych o zrotowane przykłady - poprawia odporność, ale wydłuża trening i nie gwarantuje uogólnienia na wszystkie kąty. Druga to architektury z wbudowaną geometrią: sieci grupowo równoważne (G-CNN, E(2)-equivariant), sieci cykliczne operujące na wielu orientacjach oraz przekształcenia do układów polarnych (linear-polar i log-polar), które „prostują” rotacje do przesunięć. Cel jest wspólny: by model rozpoznawał „to samo” niezależnie od orientacji, bez agresywnego dublowania danych.

Niniejsza praca skupia się na praktycznej weryfikacji tych podejść. Przygotowano zbiory obejmujące m.in. odręczne litery/cyfry, znaki drogowe (w kolorze i w odcieniach szarości) oraz syntetyczne obiekty 3D rzutowane na 2D (klocki LEGO), a następnie rozszerzono je o kontrolowane rotacje. Zaimplementowano i porównano wybrane architektury rotacyjnie inwariantne i ich warianty bazowe w **PyTorchu**, mierząc wpływ transformacji (linear-polar vs. log-polar), wyboru architektury i zakresu kątów na jakość predykcji. Obliczenia realizowano na kartach **NVIDIA GeForce RTX 3060 12 GB**, co skróciło czas trenowania i umożliwiło szeroki przegląd eksperymentów; środowisko uruchomieniowe ustandaryzowano z użyciem **Dockera** dla powtarzalności.

Celem pracy jest nie tylko pokazanie, że „da się” uzyskać odporność na rotacje, ale przede wszystkim wskazanie, **kiedy i jakim kosztem** ją osiągamy oraz które techniki przynoszą największy zysk względem klasycznych CNN-ów, jak wpływają na stabilność i szybkość uczenia, a także które konfiguracje są najpraktyczniejsze w realnych zastosowaniach (OCR, rozpoznawanie znaków, analiza obiektów technicznych). W dalszej części pracy przedstawiono podstawy, dane i augmentację, architektury, środowisko eksperymentalne, protokoły ewaluacji oraz wyniki z analizą i wnioskami.

Cel i motywacja pracy

Konwolucyjne sieci neuronowe (CNN) charakteryzują się zdolnością do analizy obrazów z zachowaniem niezmienniczości względem translacji. Jednak wciąż brakuje powszechnie uznanych architektur, które zapewniałyby inwariantność względem rotacji. Celem niniejszej pracy jest analiza skuteczności nowych rozwiązań zaproponowanych w literaturze, ukierunkowanych na zapewnienie odporności modeli na rotację danych wejściowych (np. <https://arxiv.org/pdf/2007.10588.pdf>). W ramach pracy zostały przygotowane wzbogacone zbiory danych, obejmujące m.in. zdjęcia odręcznie napisanych liter, znaków drogowych (zarówno w kolorze, jak i w odcieniach szarości) oraz obiektów 3D rzutowanych na przestrzeń 2D (klocki LEGO), rozszerzone o różnorodne rotacje obrazów.

Na podstawie tych zbiorów danych przeprowadzono implementację i ewaluację wybranych architektur rotacyjnie inwariantnych z wykorzystaniem biblioteki PyTorch. Obliczenia zostały przeprowadzone przy użyciu kart graficznych NVIDIA GeForce RTX 3060 12GB, umożliwiających przyspieszenie procesów trenowania modeli. Otrzymane wyniki zostały porównane z rezultatami klasycznych sieci konwolucyjnych, w celu oceny realnych korzyści wynikających z zastosowania rozwiązań inwariantnych względem rotacji.

Opis pracy

Praca magisterska wykorzystuje zaawansowane technologie i narzędzia wspierające badania nad rotacyjnie inwariantnymi sieciami neuronowymi oraz ich zastosowaniem w przetwarzaniu obrazów. W realizacji projektu zastosowano następujące rozwiązania technologiczne:

- **Język programowania Python** – podstawowe narzędzie do implementacji algorytmów oraz obsługi frameworków uczenia maszynowego, dzięki swojej wszechstronności i bogatemu ekosystemowi bibliotek, takich jak PyTorch i TensorFlow.

Frameworki uczenia maszynowego:

- **PyTorch** – elastyczny framework umożliwiający szybkie prototypowanie oraz trenowanie sieci neuronowych.
- **TensorFlow** – kompleksowe narzędzie do budowy, trenowania i wdrażania modeli uczenia maszynowego.
- **Modele cykliczne (CyCNN)**. W pracy zostało przyjęte podejście, w którym obraz został przemapowany do współrzędnych (ρ, φ) . Dzięki temu obrót R_α staje się przesunięciem o α po osi φ . Konwolucje zostały zastąpione warstwami cylindrycznymi (CyConv) z cyklicznym paddingiem w φ . Dla każdego filtra zostało przygotowanych n orientacji; odpowiedzi zostały złożone z dodatkową osią „orientacja”. Obrót wejścia powoduje cykliczny shift po tej osi, a pooling po orientacjach daje inwariancję względem rotacji. Został ustawiony stały środek układu polarnych, stała siatka próbkowania oraz biliniarna interpolacja; padding w φ został ustawiony na cykliczny. Implementacja została wykonana w **PyTorchu**.
- **Wykorzystanie akceleracji GPU (NVIDIA)** - obliczenia zostały znacząco przyspieszone dzięki użyciu kart graficznych NVIDIA GeForce RTX 3060 12GB, które zapewniają wydajne środowisko dla operacji intensywnych obliczeniowo. Frameworki takie jak PyTorch i TensorFlow wspierają technologie CUDA oraz Tensor, umożliwiając efektywne wykorzystanie zasobów GPU.
- **Konteneryzacja za pomocą Dockera** – narzędzie do tworzenia odizolowanych środowisk uruchomieniowych, które zapewniło łatwość replikacji środowiska oraz współdzielenia projektu, także w konfiguracjach z obsługą GPU.

Zakres tematyczny

Niniejsza praca dotyczy odporności modeli klasyfikacji obrazów na rotacje w płaszczyźnie. Skupiono się na porównaniu klasycznych architektur z ich wersjami rotacyjnie inwariantnymi oraz na wpływie przekształceń polarnych na jakość predykcji. Badania zostały przeprowadzone na obrazach 2D i rotacjach planarnych.

Ujęte w zakresie

- **Architektury modeli:** zostały zaimplementowane i porównane warianty bazowe **VGG** oraz **ResNet**, a także ich wersje cykliczne **CyVGG** i **CyResNet** (modele rotacyjnie inwariantne).
- **Przekształcenia polarne:** została oceniona użyteczność mapowania **linear-polar** oraz **log-polar** jako etapów wstępnego przetwarzania służących „prostowaniu” rotacji do przesunięć.
- **Zbiory danych:** zostały wykorzystane następujące zbiory: **MNIST** (odręczne cyfry, 28x28 przeskalowane do 32x32, grayscale), **LEGO** (syntetyczne obiekty 3D rzutowane na 2D, 96x96, grayscale), **GTSRB** (znaki drogowe, 32x32, grayscale) oraz **GTSRB_RGB** (wersja kolorowa). Zbiory zostały rozszerzone o kontrolowane rotacje; przygotowane zostały spójne podziały train/val/test.
- **Augmentacja i protokół:** zostały zdefiniowane zakresy kątów, liczba powtórzeń i podział na zbiory train/val/test (uczący/walidacyjny/testowy), z możliwością powtórzeń trenowań dla różnych losowych ziaren.
- **Środowisko i implementacja:** zostało wykorzystane **PyTorch** z akceleracją **CUDA** na kartach **NVIDIA GeForce RTX 3060 12 GB**. Środowisko uruchomieniowe zostało ustandaryzowane z użyciem **Dockera**. Przygotowane zostały skrypty w Pythonie do trenowania, testowania i ewaluacji.
- **Metryki i analiza:** została przeprowadzona ocena jakości (accuracy, macierze pomyłek), analiza stabilności (średnia/mediana/odchylenie standardowe), wpływ kąta rotacji na skuteczność oraz koszt obliczeniowy (czas trenowania, rozmiar modelu).
- **Wyniki końcowe:** zostały przygotowane rankingi modeli oraz porównania **VGG/ResNet** vs **CyVGG/CyResNet**, wraz z wnioskami praktycznymi dotyczącymi doboru architektury i przetwarzania.

Poza zakresem

- Detekcja obiektów i segmentacja - w pracy rozpatrywana jest wyłącznie klasyfikacja.
- Inwariancja względem skali, ścinania i pełnych przekształceń afinicznych - analizowana jest tylko rotacja w płaszczyźnie.
- Rotacje w geometrii 3D oraz zagadnienia widzenia stereo - pozostają poza zakresem.
- Trening na bardzo dużych korpusach z pre-treningiem self-supervised oraz szerokim AutoML/hyper-search - nie został realizowany.

- Odporność na silne zakłócenia (szum, okluzje) - poza zakresem; skupiono się wyłącznie na rotacji.

Artefakty pracy

- Repozytorium z kodem, skryptami i plikami konfiguracyjnymi (PyTorch).
- Pliki z konfiguracjami eksperymentów i opisem danych.
- Wytrenowane wagi modeli (wybrane checkpointy) oraz raporty z ewaluacji.
- Tekst pracy z dokumentacją eksperymentów i wnioskami.

Organizacja pracy

Struktura pracy została ułożona tak, by od podstaw przejść do wyników. W rozdziale **Podstawy teoretyczne** zostały zebrane pojęcia i narzędzia: CNN, inwariancja/ekwawariancja, przekształcenia polarne oraz prace pokrewne (G-CNN, CyCNN). W **Opisie zbiorów danych** zostały przedstawione MNIST, LEGO, GTSRB (gray) i GTSRB_RGB oraz sposób augmentacji (rotacje, podziały train/val/test). W **Architekturach modeli** zostały opisane warianty bazowe **VGG/ResNet** oraz wersje cykliczne **CyVGG/CyResNet**, wraz z transformacjami linear-polar / log-polar. Rozdział **Implementacja i środowisko** zawiera szczegóły techniczne: **PyTorch**, **CUDA** (RTX 3060 12 GB), **Docker**, strukturę projektu i skrypty. W **Eksperymentach** zostały zdefiniowane scenariusze, metryki i sposób ewaluacji. Dalej, w **Porównaniu wyników**, zostały zestawione modele (VGG vs CyVGG, ResNet vs CyResNet, wpływ transformacji) i omówiona stabilność/czas. Na końcu **Wnioski** zbierają najważniejsze obserwacje i wskazują kierunki dalszych badań; **Aneks** zawiera kody i dodatkowe wykresy.

Podstawy teoretyczne

Wprowadzenie do sieci konwolucyjnych (CNN)

Inwariancja translacyjna i rotacyjna

Problemy z rotacyjną inwariancją w klasycznych CNN

Przegląd literatury (np. E(2)-Equivariant CNNs, CyCNN)

Opis zbiorów danych

MNIST (cyfry odręczne)

GTSRB Gray (znaki drogowe w odcieniach szarości)

GTSRB RGB (znaki drogowe)

LEGO (obiekty 3d rzutowane na 2d)

Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów

Architektury modeli

Standardowe CNN

Rotacyjnie inwariantne sieci (np. CyResNet, CyVGG, G-CNN)

Transformacje polarne: linearpolar vs logpolar

Implementacja i środowisko eksperymentalne

Python, PyTorch

Struktura projektu

Automatyzacja: skrypty trenowania, testowania, ewaluacji

Obsługa GPU, Docker, WSL

Organizacja logów, modeli, confusion matrixów

Eksperymenty

Scenariusze trenowania/testowania (opis JSON)

Pomiar skuteczności (accuracy, macierze pomyłek)

Śledzenie metryk: średnia, mediana, odchylenie standardowe

Analiza skuteczności względem rotacji

Ranking modeli

Porównanie wyników

CyCNN vs klasyczne CNN

VGG vs CyVGG

Resnet vs CyResNet

CyVGG vs CyResNet

[...] cyresnet56 uczył się dłużej niż cyvgg19, ale dawał bardziej stabilne wyniki, w szczególności w przypadku funkcji aktywacji typu logpolar. I jak to się mówi – nie można zjeść ciastka i mieć go też²¹.

Wpływ transformacji (linearpolar vs logpolar)

Wydajność na różnych zbiorach

Wnioski

Skuteczność rotacyjnych architektur

Wnioski z automatyzacji i systematyzacji ewaluacji

Propozycje dalszych badań

Bibliografia

Aneks

Listingi kodów

Dodatkowe wykresy, tablice wyników